

Multiplicação de matrizes: otimizações e perfilamento

SSC0951 — Desenvolvimento de Código Otimizado

Atividade relativa à aula 3 • Coleta: 07/09/2026 • Entrega prevista: 09/09/2026

Integrantes e números USP: a preencher pelo grupo.

1. Objetivo e configurações

Comparar quatro implementações seriais de multiplicação de matrizes com dois modos de alocação, medindo tempo, leituras e faltas em L1 de dados, instruções de desvio e erros de predição. O enunciado define oito experimentos [1]; a operação e as transformações seguem a aula 3, páginas 14–17 [2].

Exp.	Técnica	Alocação	Laços / transformação
1	Simples	Estática	$i \rightarrow j \rightarrow k$
2	Simples	Dinâmica	$i \rightarrow j \rightarrow k$
3	Interchange	Estática	$i \rightarrow k \rightarrow j$
4	Interchange	Dinâmica	$i \rightarrow k \rightarrow j$
5	Unrolling	Estática	$i \rightarrow j \rightarrow k$; fator 2 em k
6	Unrolling	Dinâmica	$i \rightarrow j \rightarrow k$; fator 2 em k
7	Tiling	Estática	Blocos 32^3 ; $i \rightarrow j \rightarrow k$ internos
8	Tiling	Dinâmica	Blocos 32^3 ; $i \rightarrow j \rightarrow k$ internos

$$R[i][j] = \sum_{k=0}^{N-1} A[i][k] \times B[k][j]$$

Foram realizadas **80 execuções válidas**, dez por configuração. Todas utilizaram matrizes 512×512 de **float**, entradas idênticas e compilação **-O0**. A melhor média observada foi a do experimento 3: **$0,795 \pm 0,007$ s** (média $\pm$ margem do IC95%).

Os resultados descrevem esta implementação, neste computador e protocolo. Contagens menores de uma métrica não implicam, isoladamente, menor tempo de execução.

2. Ambiente, implementação e coleta

Item	Configuração registrada
Processador	AMD Ryzen 7 5700U; 8 núcleos, 16 CPUs lógicas
Caches	L1d: 32 KiB por núcleo; L2: 512 KiB por núcleo; L3: 8 MiB no total (lscpu)
Sistema	Fedora Linux 43; kernel 7.1.13-100.fc43.x86_64
Compilador	GCC 15.3.1; -std=c11 -g -O0 -Wall -Wextra -Werror
Perfilador	perf version 7.1.13-100.fc43.x86_64; pacote oficial Fedora, extraído localmente
Execução	Uma thread; afinidade na CPU lógica 2; CPU irmã 3 não isolada
Frequência	Governador: powersave; boost ativo, sem travar frequência

Memória. As matrizes estáticas têm armazenamento **static** e disposição contígua; as dinâmicas usam **float ****, vetor de ponteiros e uma alocação por linha. As funções recebem a saída e a zeram a cada chamada. O fator alocação representa também diferenças de disposição e acesso por ponteiros, e não apenas o custo de malloc.

Transformações. Interchange troca os laços j e k. Unrolling expande duas operações consecutivas do laço k e trata o resto ímpar. Tiling percorre blocos nas três dimensões, preserva i-j-k dentro deles e limita as bordas. Não se utilizam pragmas, OpenMP ou paralelismo.

Escopo. O cronômetro CLOCK_MONOTONIC mede zeramento da saída e multiplicação. O perf inicia desativado; o programa envia enable, aguarda confirmação, cronometra o cálculo e envia disable. Os contadores incluem um pequeno custo de controle e leitura do relógio ao redor do cálculo, mas excluem geração das entradas, alocação, checksum e impressão [3].

Eventos. L1-dcache-loads:u, L1-dcache-load-misses:u, branch-instructions:u e branch-misses:u foram agrupados e medidos apenas em modo usuário. Todos tiveram cobertura de 100%, sem multiplexação. Não houve alteração de permissões do kernel (perf_event_paranoid=2).

Ordem e repetições. Dez rodadas, cada uma contendo E1-E8 em ordem sorteada com semente 9512026. O piloto de oito execuções foi excluído da amostra definitiva. Cada observação usa um processo novo, sem chamada de aquecimento interna. A primeira escrita na saída integra a medição. Nenhuma observação definitiva foi descartada.

3. Validação e tratamento estatístico

Correção. As oito funções foram comparadas elemento a elemento com uma referência em double, usando matrizes densas com sinais e frações, identidade e zero. Os testes cobrem N=1, 2, 3, 10, 32, 33 e 65, blocos 32 e 7 e chamadas repetidas. AddressSanitizer e verificações de comportamento indefinido passaram para os casos registrados. Tamanhos ímpares e blocos incompletos são tratados.

Todas as 80 execuções medidas produziram checksum **33566647.392028809**. O checksum verifica a consistência da campanha; a validação matemática é feita pelos testes independentes. Os arquivos brutos, fontes utilizadas e hashes SHA-256 foram preservados.

Média e IC95%. Para cada configuração e cada uma das cinco métricas, calculou-se a média, o desvio padrão amostral (denominador n-1) e o intervalo bilateral pela distribuição t de Student [4].

$$IC95\% = \text{média} \pm t_{0,975; n-1} \times s / \sqrt{n}$$

Com n=10, o valor crítico é 2,262157. Nas tabelas, “±” indica a **margem do intervalo da média**, não o desvio padrão nem a faixa de execuções. Os limites completos e os dados com precisão integral estão em resumo.csv. Os intervalos são individuais, sem ajuste para comparações múltiplas.

Influências. Foram escolhidos previamente os conjuntos E1,E2,E3,E4 para cache e E1,E2,E5,E6 para branch. Em cada análise 2², A é a técnica (-1 simples, +1 otimizada), B é a alocação (-1 estática, +1 dinâmica), e AB é a interação [5].

$$y = q_0 + q_A A + q_B B + q_{AB} AB + \text{erro}$$

Os coeficientes são obtidos pelos contrastes das quatro médias divididos por 4. Cada efeito é 2q. Com r=10 réplicas por célula: $SQ_A = 4rq_A^2$, analogamente para B e AB; SQ_{erro} é a soma dos quadrados dos desvios dentro de cada célula.

$$SQ_{\text{total}} = SQ_A + SQ_B + SQ_{AB} + SQ_{\text{erro}}$$

A influência com réplicas é $100 \times SQ_{\text{fator}} / SQ_{\text{total}}$. Também é apresentada a normalização somente entre as quatro médias: $100 \times q_{\text{fator}}^2 / (q_A^2 + q_B^2 + q_{AB}^2)$. Esta segunda forma soma 100% entre os fatores, mas omite a variação entre repetições. Percentual de influência não é percentual de melhoria de desempenho.

As contas foram executadas em Python e verificadas por decomposição da soma de quadrados e mínimos quadrados em um exemplo de coeficientes conhecidos. O arquivo de apoio r.txt estava vazio durante a preparação; não se atribui esta implementação a um script da disciplina não lido.

4. Comparação dos tempos dos oito experimentos

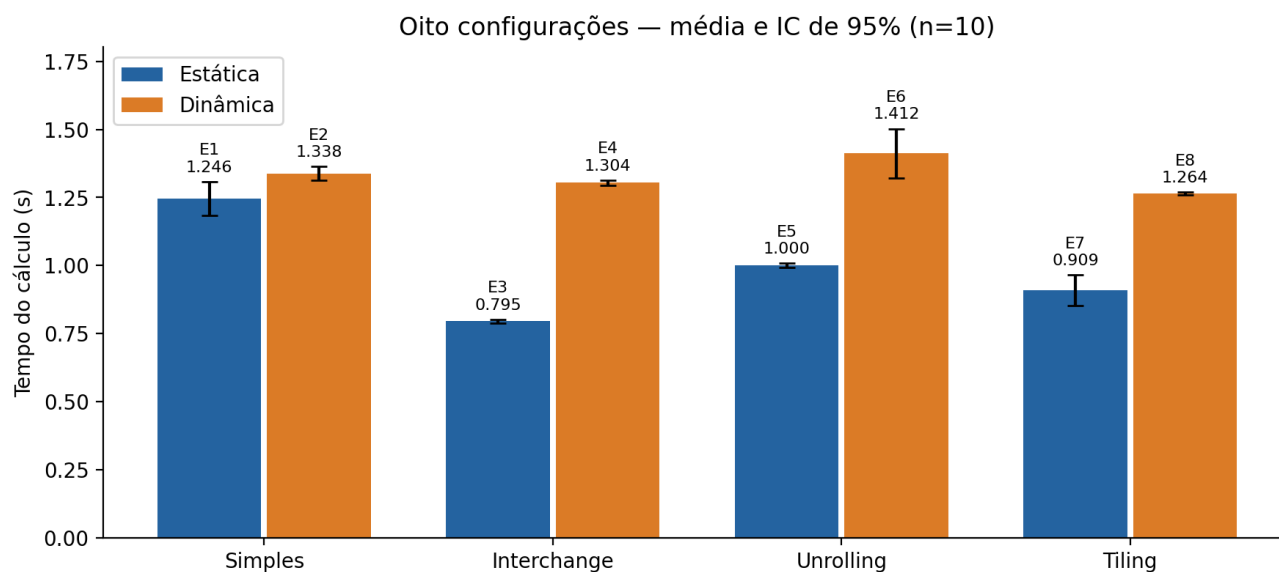


Figura 1 — Médias e intervalos de confiança de 95% para o tempo do cálculo.

Exp.	Técnica / alocação	Tempo (s), média ± IC95%	Razão simples / versão
1	Simple / estática	1,246 ± 0,062	1,000×
2	Simple / dinâmica	1,338 ± 0,025	1,000×
3	Interchange / estática	0,795 ± 0,007	1,567×
4	Interchange / dinâmica	1,304 ± 0,010	1,026×
5	Unrolling / estática	1,000 ± 0,008	1,247×
6	Unrolling / dinâmica	1,412 ± 0,091	0,948×
7	Tiling / estática	0,909 ± 0,057	1,371×
8	Tiling / dinâmica	1,264 ± 0,005	1,059×

O interchange estático (E3) reduziu o tempo médio em **36,2%** em relação a E1, razão de 1,57×. As médias estáticas seguiram a ordem $E3 < E7 < E5 < E1$.

Na alocação dinâmica, E8 teve a menor média: $1,264 \pm 0,005$ s, redução de 5,6% ante E2. E6 teve média 5,5% maior que E2. Seus intervalos se sobrepõem; essas razões são descritivas e não constituem testes de significância.

O custo de carregar ponteiros por linha e a geração de endereços em -O0 podem contribuir para as diferenças entre versões estáticas e dinâmicas. O perfilamento obtido não isola esses mecanismos nem prova uma causa única.

5. Leituras e faltas na cache L1 de dados

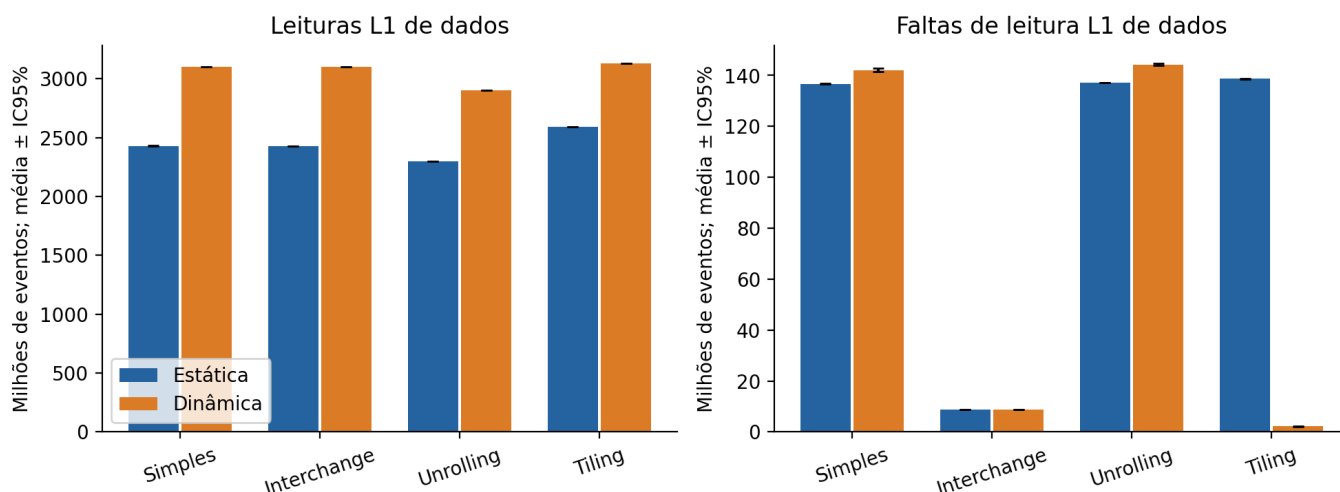


Figura 2 — Contagens de cache para as oito configurações; barras de IC95%.

Exp.	L1 loads (milhões), média ± IC95%	L1 misses (milhões), média ± IC95%
1	2.430,294 ± 1,395	136,723 ± 0,074
2	3.101,304 ± 0,175	142,076 ± 0,625
3	2.427,685 ± 0,126	8,700 ± 0,012
4	3.098,745 ± 0,079	8,809 ± 0,019
5	2.299,254 ± 1,613	137,130 ± 0,097
6	2.902,800 ± 0,354	144,246 ± 0,400
7	2.588,808 ± 0,225	138,738 ± 0,134
8	3.128,553 ± 0,103	2,199 ± 0,092

Interchange diminuiu as faltas de L1 de 136,72 para 8,70 milhões na alocação estática (**93,6%**) e de 142,08 para 8,81 milhões na dinâmica (**93,8%**). A ordem i-k-j percorre linhas de B e da saída no laço interno, coerente com maior localidade.

O total de loads permanece elevado. Esses eventos abrangem as leituras executadas pelo código, inclusive acessos a variáveis e ponteiros em -O0; não equivalem apenas ao número de elementos matemáticos de A e B.

O tiling apresentou forte diferença: E7 teve 138,74 milhões de faltas e E8, 2,20 milhões. Uma hipótese é conflito de conjuntos causado pelo passo de 2.048 bytes entre linhas estáticas e pela ordem interna i-j-k; as linhas dinâmicas têm disposição distinta. Seriam necessários outros tamanhos, blocos ou medidas de endereços para confirmar. Não se conclui que tiling sempre melhora cache.

6. Instruções de desvio e erros de predição

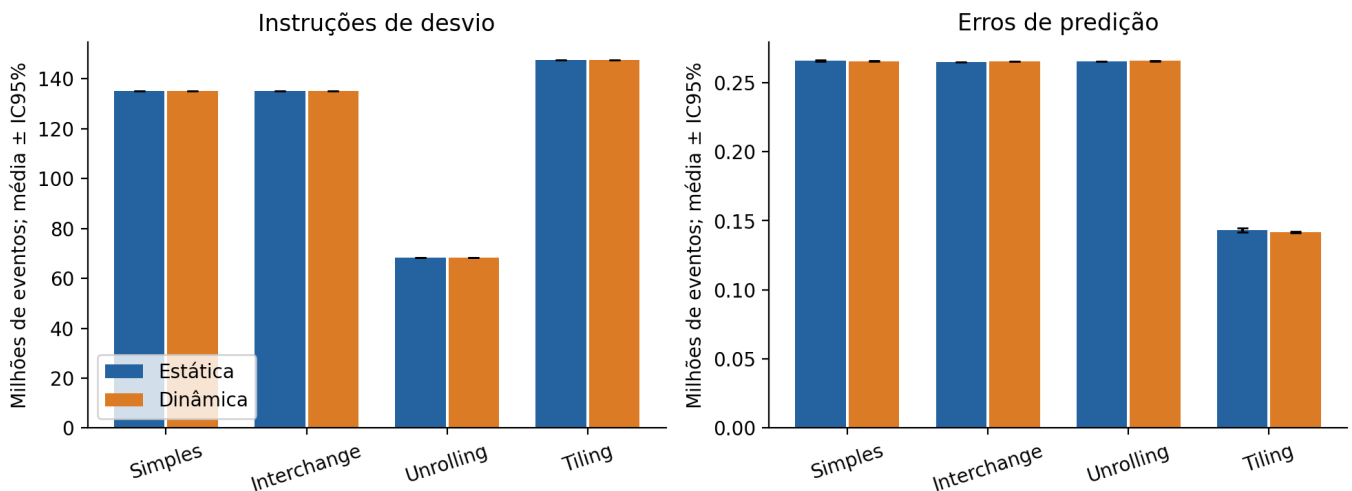


Figura 3 — Contagens de branch; o número de desvios e o de erros são métricas distintas.

Exp.	Branch instr. (milhões), média ± IC95%	Branch misses (milhares), média ± IC95%
1	135,27209 ± 0,00007	265,920 ± 0,309
2	135,27194 ± 0,00003	265,594 ± 0,227
3	135,27163 ± 0,00004	264,771 ± 0,093
4	135,27190 ± 0,00001	265,399 ± 0,115
5	68,42512 ± 0,00001	265,319 ± 0,108
6	68,42533 ± 0,00012	265,711 ± 0,325
7	147,49065 ± 0,00038	143,274 ± 1,561
8	147,49060 ± 0,00002	141,954 ± 0,464

O unrolling reduziu as instruções de desvio de cerca de 135,272 para 68,425 milhões: **49,42%** na alocação estática e 49,42% na dinâmica. Isso é coerente com metade das iterações de controle do laço k, preservando as multiplicações.

Os erros de predição de E1,E2,E5,E6 ficaram próximos de 265–266 mil. Assim, diminuir quase pela metade o total de desvios não reduziu os erros na mesma proporção. A razão entre as médias de misses e instructions passou de 0,197% em E1 para 0,388% em E5; essa taxa é descritiva, sem IC específico.

Tiling aumentou o total de instruções de desvio devido aos laços de bloqueio, mas apresentou menos erros de predição. A mudança no padrão de controle pode explicar parte do comportamento. Como cache, acessos e controle mudam conjuntamente, uma só contagem não explica o tempo final.

7. Influência dos fatores e da interação

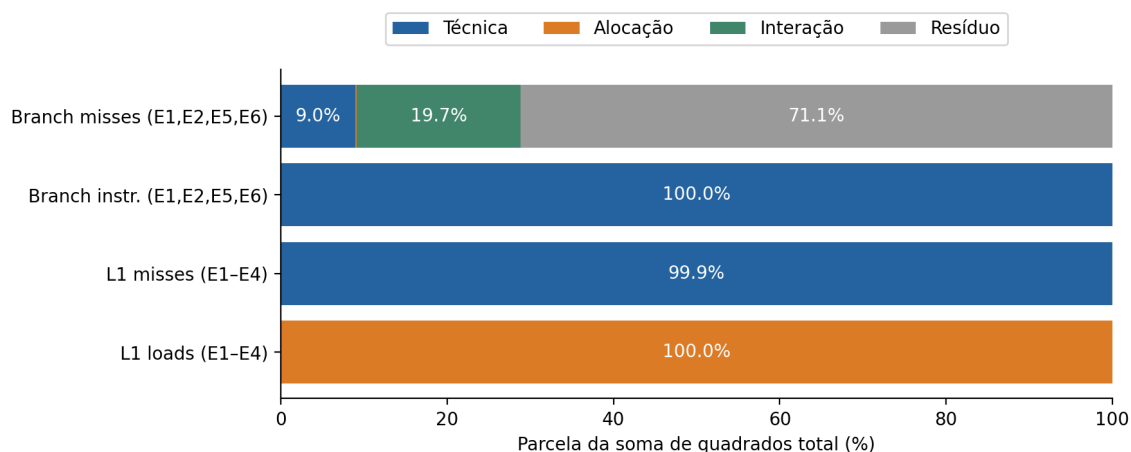


Figura 4 — Decomposição com réplicas: técnica, alocação, interação e resíduo.

Métrica	Técnica (%)	Alocação (%)	Interação (%)	Resíduo (%)
L1 loads	0,001	99,998	0,000	0,001
L1 misses	99,912	0,044	0,040	0,004
Branch instr.	100,000	0,000	0,000	0,000
Branch misses	8,952	0,170	19,739	71,138

Tabela 5 — Percentuais da soma de quadrados total das 40 observações de cada subconjunto. Valores exibidos como 0,000% podem ser positivos abaixo da precisão de apresentação.

Métrica	Técnica (%)	Alocação (%)	Interação (%)
L1 loads	0,001	99,999	0,000
L1 misses	99,916	0,044	0,040
Branch instr.	100,000	0,000	0,000
Branch misses	31,018	0,589	68,393

Tabela 6 — Normalização entre as quatro médias, sem componente de erro. O denominador difere do usado na Tabela 5.

Cache (E1-E4). A alocação explica 99,998% da variação total em loads; a técnica explica 99,912% em misses. Em média sobre as duas técnicas, mudar de estática para dinâmica acrescentou 671,03 milhões de loads. Interchange reduziu, em média sobre as alocações, 130,64 milhões de misses.

Branch (E1,E2,E5,E6). A técnica responde por praticamente 100% da variação nas instruções de desvio. Já em branch misses, **71,14% é resíduo entre repetições**. O efeito médio do unrolling foi de apenas -241,7 erros por execução. A influência da interação parece maior quando o erro é omitido; não se deve interpretar esse percentual como evidência forte de ganho.

8. Variabilidade, limites e conclusões

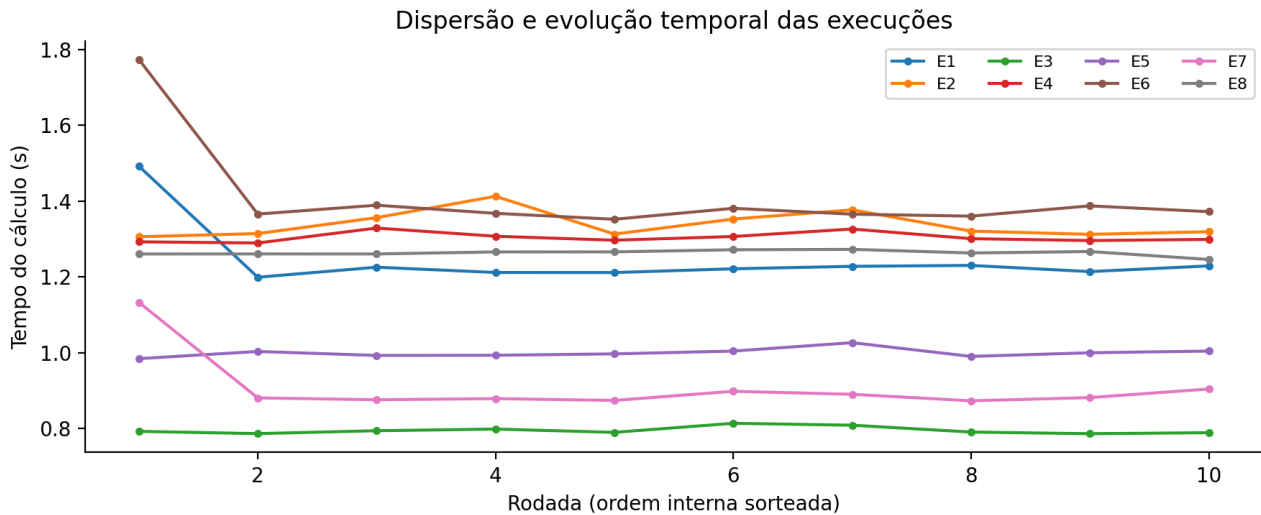


Figura 5 — As primeiras observações de alguns experimentos foram mais lentas e foram preservadas.

A afinidade reduz migrações entre núcleos, mas não elimina disputa com processos do sistema, uso da CPU irmã, variação de frequência ou efeitos térmicos. A coleta ocorreu em um computador de uso geral, com boost ativo e sem isolamento do núcleo. A ordem sorteada distribui parte desses efeitos, mas não garante independência perfeita.

Os ICs t pressupõem observações suficientemente independentes e distribuição das médias adequadamente aproximada; com dez repetições, assimetria e variações de estado da máquina podem afetar essa aproximação. E1, E6 e E7 tiveram intervalos mais largos. Não foram removidos valores extremos, nem feitas repetições seletivas.

N=512 gera três matrizes de aproximadamente 3 MiB no total, acima da L1d e L2 de um núcleo. BLOCO=32 e fator de unrolling 2 foram fixados, sem busca pelo melhor ajuste. A dimensão é potência de dois e pode acentuar conflitos de cache. Os resultados não descrevem todos os tamanhos e blocos possíveis.

As medições incluem zeramento e primeira escrita da saída; excluem o tempo de malloc/free e usam apenas eventos de usuário. Portanto, a comparação de alocação avalia principalmente os acessos e a disposição das matrizes, não o custo total de criação de estruturas. Os percentuais de influência dependem dos níveis e subconjuntos escolhidos.

Conclusões. Interchange estático apresentou o menor tempo médio e reduziu fortemente as faltas de L1. Unrolling diminuiu as instruções de desvio em aproximadamente metade, mas isso não garantiu melhora no caso dinâmico. Tiling dependeu fortemente da disposição em memória. A análise com réplicas mostrou que a variação em branch misses foi dominada pelo resíduo, exigindo cautela para atribuir efeitos aos fatores.

9. Reprodutibilidade e referências

Os dados desta versão estão em **Ex1/resultados/coleta_2026-09-07/**. O diretório contém `medicoes.csv` (80 linhas), `metadados.json` (máquina, versões, ordem e hashes), `brutos/` (saída original do perf e do programa) e `fontes/` (snapshot dos arquivos usados na coleta).

Em análise/: `resumo.csv` contém as 40 combinações experimento/métrica com média, desvio, limites do IC, mínimo, máximo e CV; `influencias.csv` e `influencias.json` guardam os efeitos e somas de quadrados; `graficos/` contém PNG e PDF vetorial. O piloto está em outro diretório e não entra nos cálculos.

Os programas de coleta, análise e geração do relatório estão em `scripts/`. Para reproduzir a análise dos dados existentes, a partir de Ex1:

```
../../ferramentas/venv/bin/python scripts/analisar.py resultados/coleta_2026-09-07
../../ferramentas/venv/bin/python scripts/relatorio.py resultados/coleta_2026-09-07
```

Uma nova coleta deve usar um diretório novo, após compilar com TAMANHO=512 e BLOCO=32. O coletor recusa sobrescrever uma coleta existente e exige pelo menos dez rodadas no modo definitivo. Um arquivo de dados incompleto, checksum divergente, contador indisponível ou multiplexado interrompe a validação.

A instalação do perf no sistema exigiu senha; utilizou-se o pacote Fedora `perf-7.1.13-100.fc43.x86_64`, com assinatura verificada, extraído em `.ferramentas/perf/`. Não foram alterados pacotes do sistema ou permissões de contadores. O ambiente Python local usa NumPy 2.2.6 compatível com SciPy 1.14.1.

Referências

- [1] SSC0951. Atividade relativa à aula 3 (Profiling). Arquivo local `Atividade_1.pdf`. Enunciado, métricas, repetições, IC95% e entrega.
- [2] BRUSCHI, Sarita Mazzini. 3ª aula — Técnicas básicas para otimização de código serial / Profiling. Arquivo local `3aAula_Profiling.pdf`, páginas 14–17.
- [3] Linux perf. `perf-stat(1)`: controle enable/disable, --delay e grupos de eventos. man7.org/linux/man-pages/man1/perf-stat.1.html. Consulta: 07/09/2026.
- [4] NIST/SEMATECH. Confidence Limits for the Mean. *NIST e-Handbook*, seção 1.3.5.2. Consulta: 07/09/2026.
- [5] NIST/SEMATECH. The two-way ANOVA. *NIST e-Handbook*, seção 7.4.2.7. Consulta: 07/09/2026.

Identificação: Integrantes e números USP: a preencher pelo grupo.